



Mission - Option B - Cas fictif



Vous êtes en stage au sein d'une entreprise spécialisée dans le développement d'applications mobiles. Jusqu'à présent, vous avez travaillé en tant que soutien de l'équipe dans la création d'applications.

Il est temps de passer à la conception d'une application par vous-même !

Cette application sera développée pour le compte de l'entreprise automobile Vitesse.



Elle aura pour but d'optimiser la gestion des candidats par le service des ressources humaines.

Les RH pourront ainsi créer, modifier et ajouter des candidats à leur liste, tout en ayant la possibilité de les marquer en tant que favoris.

Svetlana, la Product Owner, vous a fourni [les fonctionnalités nécessaires](#) pour le projet, ainsi que des [wireframes](#) et la palette de couleurs (disponible dans le Kanban).

Vous entrez en contact avec le Lead Développeur, Sam.

De : Sam

À : Moi

Objet : Tech radar et base de données des candidats

Salut,

J'espère que tu vas bien !

Les User Stories semblent compatibles avec notre DoR (Definition of Ready) donc tu peux te lancer sans soucis. N'oublie pas que **les tests unitaires** participent à la DoD (Definition of Done) donc fais-les dès que possible.

Concernant la stack technique, tu peux utiliser ce que tu veux tant que ça reste compatible avec ce qui est référencé dans [le Tech Radar de l'entreprise](#).

Je n'ai malheureusement pas de base de données à te fournir, mais en parcourant le KANBAN, tu trouveras toutes les informations nécessaires à la structuration de celle-ci.

N'hésite pas si tu as des questions.



Vous avez toutes les clés en main pour réaliser votre première application Android seul !

Étapes

Étape 1 - Prenez connaissance du projet

La première étape est de se définir des jalons. Ce projet est plus imposant que les précédents et aucun code n'est fourni au départ. Vous possédez cependant tout ce qu'il faut pour l'appréhender sereinement :

- l'ensemble des tâches à réaliser pour mener à bien le projet via le KANBAN ;
- la stack technique autorisée à travers le tech radar.

Bien que le KANBAN soit très détaillé, il est très orienté vers le fonctionnel de l'application à travers les nombreuses User Stories. Essayez de déterminer les tâches techniques qui sont parfois un prérequis pour implémenter une User Story.

- Quelle est la structure de la base de données ?
- Quelle(s) table(s) vais-je devoir créer ? supplémentaires ?
- Avec quels champs ?
- S'il y a plusieurs tables, quelles sont les relations entre celles-ci ?

Recommandations pour cette étape

Papier et crayons seront vos meilleurs amis pour cette étape.

L'idée ici est de prendre connaissance du sujet et, en quelque sorte, de le planifier.

L'ampleur du projet peut faire peur, mais en réalité, chaque tâche, prise séparément, n'est pas si difficile à implémenter.

Au cours des étapes suivantes, n'hésitez pas à revenir sur ce que vous aviez décidé : plus vous avancerez dans le projet, plus il sera clair dans votre tête, et plus vous aurez de chance de vous rendre compte que vous avez oublié ou mal compris telle ou telle partie.

Quelques définitions

Le projet contient plusieurs définitions avec lesquelles vous n'êtes peut-être pas familier. Aussi, prenez le temps nécessaire pour les comprendre.

Un tech radar est un outil de veille technologique permettant de visualiser les technologies et les méthodologies utilisées par une entreprise. Chaque technologie est positionnée en fonction de son niveau de maturité, de son adoption et de son potentiel d'impact.

L'intérêt d'un tech radar pour une entreprise est multiple. Il permet notamment :

- de suivre les tendances technologiques et de s'assurer que l'entreprise utilise les technologies les plus adaptées à ses besoins ;
- de prendre des décisions éclairées en matière d'investissements technologiques ;
- d'améliorer la communication et la collaboration entre les équipes techniques.

Un tech radar est un outil précieux pour toute entreprise qui souhaite rester compétitive dans un environnement technologique en constante évolution.

Le Tech Radar proposé dans le cadre de ce projet classe les technologies en 5 statuts :



- **Watch** : Cette technologie ne doit pas être utilisée en production, mais peut l'être sur des POC pour comprendre l'intérêt pour l'entreprise ;
- **Hold** : Cette technologie est présente en production sur des projets, mais elle ne doit pas être utilisée dans le cadre de nouveaux développements ;
- **Forbidden** : L'usage de cette technologie est interdit dans l'entreprise.

La Definition of Ready (DoR) est une liste de critères qu'un Product Backlog Item (PBI) (User Story, EPIC, tâche, bug, etc.) doit remplir pour être considéré comme prêt à être implémenté. Elle permet de s'assurer que le PBI est suffisamment défini et complet pour que l'équipe de développement puisse le prendre en charge.

La Definition of Done (DoD) est une liste de critères qu'un PBI doit remplir pour être considéré comme terminé. Elle permet de garantir que le PBI répond aux besoins des utilisateurs et qu'il est prêt à être mis en production.

Étape 2 - Initialisez la structure de votre projet



Avant de vous attaquer à l'implémentation des premiers tickets qui constituent le KANBAN, il est important de s'attaquer aux prérequis techniques, à savoir :

- **créer le dépôt git** qui accueillera le code source de votre application ;
- **créer le projet Android**, correctement configuré, en ciblant la dernière version stable du SDK Android qui accueillera le code des différentes fonctionnalités ;
- **mettre en place les bases d'une architecture logicielle** compatible avec le Tech Radar de l'entreprise.

Ces étapes constituent un échauffement avant la prochaine étape. Une fois mises en place, l'implémentation des fonctionnalités demandées sera largement facilitée !

Recommandations pour cette étape

Avant de passer à l'étape suivante, assurez-vous que le lien entre votre code et votre dépôt Git est correctement configuré pour éviter les mauvaises surprises.

Assurez-vous également que la base de votre architecture logicielle compile. Elle se doit d'être une base solide pour la suite et est la garantie d'un projet plus facile à réaliser. Relisez les cours si nécessaire et revoyez vos exercices si vous avez des difficultés. À ce stade, ce qui est important, c'est de comprendre en profondeur ce que vous faites, comment et pourquoi.

Étape 3 - Implémentez les fonctionnalités



Au préalable, la mise en place de Git doit être prête et votre architecture mise en place. Il est temps de véritablement commencer à implémenter les différentes tâches demandées : icône, palette couleurs, affichage, ajout et modification des candidats, etc.

Pour implémenter les différentes tâches demandées, ne vous limitez pas à l'écriture de code, prenez également le temps de le documenter.

Recommandations pour cette étape

Si vous ne savez pas par où commencer, juste... commencez !

Les tâches proposées dans le KANBAN sont suffisamment fines pour être réalisées de manière autonome. Vous n'êtes pas obligé de les réaliser dans l'ordre proposé dans le KANBAN. Cependant, si vous faites ce choix, assurez-vous de bien comprendre les potentielles dépendances entre certaines tâches. N'oubliez pas de faire des commits réguliers de votre code au fur et à mesure de votre avancée.



N'hésitez pas également à relire le cours "[Gérez vos données localement pour avoir une application 100 % hors-ligne](#)" ainsi que vers la [documentation officielle](#).

Si vous éprouvez des difficultés à implémenter les appels réseaux, référez-vous à ce que vous avez déjà accompli dans le cadre du projet "[Implémentez l'accès à des données externes pour une application en Kotlin](#)". Vous pouvez aussi relire le cours "[Récupérez et affichez des données distantes](#)" dédié à cette problématique.

Pour la sélection de la photo d'un candidat, s'il existe de nombreuses bibliothèques tierces permettant de couvrir ce besoin, orientez-vous plutôt vers le récent [sélecteur de photos](#) proposé directement par Google. Ce sélecteur de photos expose notamment l'Uri de la photo sélectionnée. Cela vous sera utile pour sauvegarder la photo dans la base de données.

Comme vous l'avez remarqué en parcourant les différentes tâches du KANBAN, l'application doit supporter deux langues : le français et l'anglais. Si vous ne savez pas par quel bout attaquer ce sujet, vous pouvez vous tourner vers la documentation Android qui explique comment [assurer la compatibilité d'une application avec différentes langues et cultures](#).

Finalement, c'est peut-être la première fois que vous êtes confronté à la création d'un menu. Une fois de plus, n'hésitez pas à lire [la documentation officielle](#) à ce sujet.

Étape 4 - Réalisez les tests

Si vous reprenez le mail de Sam, vous remarquerez la recommandation quant à l'écriture de tests, *a minima*, unitaires pour ce projet.

Même si cette tâche n'apparaît pas dans le Kanban, il faut la faire car elle est indispensable pour un développeur. En effet, comme le souligne Sam dans son email, les tests font partie de la DoD de l'entreprise. Aussi, pour être considérée comme terminée, une tâche doit répondre à la DoD et donc être couverte par des tests.

Recommandations pour cette étape

Testez ! Ne soyez pas avarés sur les tests. Testez tout ce que vous pouvez.

Si l'on reproche souvent aux développeurs le manque de tests lors de l'écriture d'un logiciel, le contraire est rare.

On ne vous reprochera jamais d'avoir essayé de couvrir au maximum votre code. À noter que dans certaines entreprises, pour être accepté, le code produit par les développeurs doit être couvert par un pourcentage minimum de tests. Pour les entreprises les plus exigeantes, la couverture de test doit être au minimum de 80 %. Si vous vous demandez comment connaître la couverture de test de votre code, sachez qu'il existe plusieurs solutions :

- directement [dans Android Studio](#) ;
- via la [bibliothèque tierce JaCoco](#) (Java Code Coverage). Si cette bibliothèque est à destination du langage Java, elle est parfaitement compatible avec Kotlin ;
- via la [bibliothèque tierce Kover](#). Cette bibliothèque est à destination du langage Kotlin, mais reste jeune et encore en plein développement.

Dans votre cas, il n'est pas nécessaire d'aller jusqu'à implémenter un de ces outils pour analyser la couverture de test de votre code. Contentez-vous de viser la demi-douzaine de tests unitaires.

Dernière étape - Vérifiez votre travail

Pour vérifier que vous n'avez rien oublié dans la réalisation de vos livrables, téléchargez et complétez [la fiche d'auto-évaluation](#).

Vous pourrez en discuter avec votre mentor.



Précédent

Suivant